

PROFILE

김용우

개발 포트폴리오

업무를 이해하고, 데이터와 시스템을 연결합니다.

BACKEND ENGINEERING · DATA PIPELINE · AI WORKFLOW

고객사 광고 운영 시스템의 장기 유지보수부터
통합 플랫폼 구축, 개발 워크플로 자동화와 이미지 검색 기술 검증까지.

PROJECT	주요 경험	PAGE
01 · AI 코드리뷰 / CI/CD	AI를 개발 업무에 연결하고 배포·복구 자동화	03
02 · 통합 광고 플랫폼	PL · 인벤토리 가공 · 구매 트랜잭션	07
03 · 이미지 유사도 검색	모델·검색 방식 검토 · 벡터 색인 · API	13
04 · 광고 로그 분석	대용량 데이터 처리 · 배치 운영	18

김용우 Backend Developer

PROFILE

경험과 역할

고객 요구사항을 구현하고, 운영 과정의 문제를 개선해 왔습니다.

실서비스 개발과 운영

2015년부터 고객사별 광고 운영 청약 시스템을 개발·운영했습니다. 광고, 소재, 타겟팅, 판매 청약 업무를 다루며 고객과 직접 요구사항과 우선순위를 조율했습니다. 이후 통합 광고 플랫폼에서는 PL과 핵심 비즈니스 로직 개발을 맡았습니다.

프로젝트 / 기간	역할 / 기여도
영상 광고 운영 청약 시스템 유지보수 및 고도화 2015.11 - 2025.05	풀스택 개발 · 서비스 운영 및 고도화 / 70%
대용량 광고 로그 분석 시스템 2024.03 - 2024.05	데이터 처리 파이프라인 설계 · 백엔드 개발 / 100%
통합 광고 인벤토리 운영 및 광고 관리 시스템 2025.05 - 2026.03	PL · 핵심 비즈니스 로직 개발 / 70%
AI 모델 기반 이미지 유사도 검색 PoC 2026.04 - 2026.05	AI 검색 시스템 설계 · 백엔드 개발 / 100%
AI 기반 코드리뷰 및 CI/CD 배포 자동화 2026.06 - 2026.07	CI/CD 설계·개발 · AI 코드리뷰 자동화 / 100%

장기 운영 경험을 신규 개발로 연결

- 레거시 리팩토링과 쿼리·인덱스 튜닝으로 평균 응답시간을 300ms에서 80ms 수준으로 개선
- Python/Django 기반 점진적 마이그레이션 및 관리자 UI/UX 리뉴얼
- AWS 기반 모니터링·인프라 운영과 트래픽 급증·장애 대응

수록 프로젝트 근무처: (주)니트머스

01 / AI CODE REVIEW & CI/CD

AI 기반 코드리뷰 및 CI/CD 배포 자동화

리뷰부터 테스트, 승인 배포, 장애 복구까지 하나의 개발 흐름으로 연결했습니다.

2026.06 - 2026.07 · 기여도 100%

CI/CD 설계 및 개발 / AI 코드리뷰 자동화

15분 → 1분

개발자 배포 개입 시간

타 개발팀 확대

팀 내 AI 리뷰 적용 이후

문제

환경별 배포 스크립트를 개발자가 직접 실행하면서 배포 대상 확인, 버전 확인과 결과 공유에 수작업이 필요했습니다. 장애 발생 시에도 이전 버전을 확인하고 다시 배포해야 했습니다.

직접 맡은 범위

- Claude CLI 기반 PR 코드리뷰와 프롬프트 규칙 설계
- Bitbucket Pipelines 기반 테스트·이미지 빌드·환경별 배포
- AWS Lambda와 SES를 연결한 이메일 승인 및 결과 공유
- Health Check 실패 시 자동 롤백과 선택 버전 재배포

자동화의 경계

반복 작업은 자동화하고, AI 리뷰의 최종 판단과 배포 시점 결정은 사람이 맡도록 구성했습니다.

사용 기술

Python · Bitbucket Pipelines · Claude CLI · Docker / Compose · AWS Lambda · AWS SES · pytest · Linux / Shell

01 / AI CODE REVIEW & CI/CD

PR 리뷰를 업무 흐름에 통합

AI의 분석 결과를 개발자가 실제 검토하는 PR에 전달했습니다.

01 PR 등록 / 업데이트

Bitbucket Webhook 수신

**02 변경 내용 수집**

Bitbucket API로 PR Diff 수집

**03 Claude CLI 리뷰**

Diff 규모에 따라 전체 또는 파일별 분할 분석

**04 결과 전달**

Bitbucket PR 코멘트 자동 등록 + SES 이메일

**05 팀원 검토**

리뷰 의견 확인 후 수정·머지 판단

리뷰 품질을 위한 처리

판단 지점	적용 방식
리뷰 범위	불필요한 파일을 제외하고 변경 Diff를 중심으로 분석
컨텍스트 크기	Diff 규모에 따라 전체 리뷰와 파일별 분할 리뷰 적용
프롬프트 규칙	근거가 부족하거나 사소한 의견을 줄이고 오류·장애 가능성·보안 이슈에 집중
전달 경로	PR에서 결과를 확인할 수 있도록 코멘트 등록, 이메일 병행

업무 적용 결과

팀 내 코드리뷰에 적용한 뒤 다른 개발팀까지 확대했습니다. AI의 의견을 참고하되 최종 반영 여부는 개발자가 판단합니다.

01 / AI CODE REVIEW & CI/CD

테스트부터 배포·롤백까지

실패 시 다음 단계로 넘어가지 않고, 배포 버전과 복구 경로를 명확히 했습니다.

01 develop Push / master Merge

환경별 Bitbucket Pipeline 실행

**02 pytest Test Gate**

테스트 실패 시 빌드·배포 중단

**03 Docker 이미지 빌드 / Registry Push**

환경·빌드 번호·시각을 포함한 태그 생성

**04 빌드 완료 이메일 / 배포 승인**

SES 이메일의 배포 링크 → Lambda → Bitbucket API

**05 서버 배포 / Health Check**

SSH와 Docker Compose로 적용, 실패 시 이전 이미지 롤백

버전 및 결과 관리

- dev/prod 환경을 분리하고 배포할 이미지를 태그로 식별
- 배포 완료 후 적용 버전과 결과를 이메일로 공유
- 이전 버전 자동 롤백과 지정한 버전의 재배포 지원

성과의 기준

약 15분에서 1분으로 줄어든 시간은 개발자가 배포에 직접 개입하는 시간입니다. 이미지 빌드와 서버 배포의 전체 실행 시간을 의미하지 않습니다.

설계 판단과 운영 결과

배포 빈도와 광고 운영 일정에 맞춰 자동화 범위를 정했습니다.

설계 선택	이유와 고려 사항
이메일 승인 배포	광고 운영 일정에 따라 배포 시점을 판단할 수 있도록 사람의 승인을 유지했습니다. 빌드 완료와 서버 반영 시점을 분리했습니다.
Lambda 기반 배포 트리거	주 1~2회 수준의 배포 트리거를 위해 상시 서버를 운영하지 않도록 구성했습니다. AI 리뷰 서버는 별도로 운영합니다.
환경·빌드 번호·시간 기반 태그	어느 환경의 어떤 이미지인지 식별하고, 롤백 대상 버전을 지정할 수 있도록 했습니다.
테스트 게이트 + Health Check	배포 전 테스트 실패와 배포 후 서비스 이상을 서로 다른 단계에서 확인하도록 했습니다.

변경 전후

항목	변경 전	변경 후
개발자 개입	약 15분	약 1분
배포 대상 선택	수동 스크립트 선택	브랜치 기반 환경 분리
배포 내역 공유	배포자가 직접 공유	이메일 자동 발송
리뷰 결과 전달	팀원 간 확인	AI 1차 리뷰를 PR 코멘트와 이메일로 전달
복구	버전 확인 후 수동 재배포	자동 롤백 + 선택 버전 재배포

확장 방향: 제어 가능한 도구 호출과 검증 루프

LLM 리뷰는 제한된 Diff와 결과의 비결정성으로 근거가 부족한 판단을 내릴 수 있습니다. 허용된 소스 조회·테스트·정적 분석 도구만 호출하고 반복 횟수와 실행 범위를 제한하면, 실행을 통제하면서 결과를 리뷰 근거로 남길 수 있습니다. 코드 변경의 최종 판단은 개발자가 담당합니다.

02 / ADVERTISING PLATFORM

통합 광고 인벤토리 운영 및 광고 관리 시스템 신규 구축

여러 플랫폼의 인벤토리를 통합하고 광고 구매부터 운영·송출 연동까지 관리했습니다.

2025.05 - 2026.03 · 기여도 70%

PL / 핵심 비즈니스 로직 개발

20% 이상

기존 판매풀 대비 판매 가능 인벤토리 확대

PL + 개발

고객 요구사항 조율과 핵심 로직 구현

문제와 구축 범위

플랫폼별로 분산된 예측 인벤토리를 광고 상품의 판매 기준으로 변환하고, 구매와 차감 결과를 운영 데이터로 연결해야 했습니다. 캠페인·상품·유닛·타겟팅·소재를 함께 관리하는 신규 시스템을 구축했습니다.

직접 맡은 범위

- 고객사 요구사항 정의, 개발자 간 작업 분배 및 일정 관리
- 프로젝트 구조와 주요 DB 설계, 관리자·프런트 공통 뷰와 템플릿 설계
- 외부 예측 데이터 가공, 판매 인벤토리 생성, 광고 구매·차감 로직
- 광고 메타데이터 및 소재 인코딩 연동 API 개발

사용 기술

Python · Django · MySQL · JavaScript / jQuery · Docker · AWS EC2 / S3 / Route53 · Sentry

02 / ADVERTISING PLATFORM

업무별 책임을 분리한 구조

공통 업무 로직을 서비스 계층에 두고 관리자/사용자 화면에서 활용했습니다.

01 화면 / 요청 처리

Django Templates · JavaScript · 관리자/프런트 뷰

**02 업무 서비스**

구매 · 인벤토리 차감 · 상태 관리 · 엑셀 생성

**03 데이터 접근**

Django ORM · 대량 처리를 위한 SQL

**04 업무 데이터 저장**

MySQL · 캠페인 / 구매유닛 / 광고 / 소재 / 인벤토리

구조 설계에서 고려한 점

대상	구성 방향
관리자 / 프런트	공통 뷰·템플릿을 활용하고 사용자 전용 화면을 분리
구매 업무	구매 데이터 생성과 인벤토리 처리의 역할을 서비스로 분리
데이터 처리	ORM을 활용하면서 대량 생성·갱신에는 Bulk 처리와 SQL 적용
변경 관리	Git/PR 규칙, 코드 컨벤션, DB 명명 규칙을 정의

PL 역할

요구사항을 이슈 단위로 정리하고 개발자별 작업과 일정을 조율했습니다. 설계와 핵심 업무 구현을 함께 맡아 기획 요구사항을 데이터·서비스 구조로 연결했습니다.

02 / ADVERTISING PLATFORM

예측 데이터를 판매 인벤토리로

외부 데이터와 내부 상품의 기준 차이를 단계별 변환으로 처리했습니다.

01 외부 예측 인벤토리

프로그램·플랫폼·일별 예측 데이터 수신

**02 표준 인벤토리**

파트 타입 매핑 · 그룹화 · 합산 · 확인 및 보정

**03 판매 인벤토리**

상품·플랫폼 기준 변환 · 일별 비율 분배

**04 광고 구매**

판매 가능 잔여량 확인 및 인벤토리 차감

핵심 처리

- 외부 데이터의 분류를 내부 상품·플랫폼 기준으로 매핑
- 일별 예측 비율과 그룹 타겟팅 비중을 반영한 분배
- 소수점 반올림으로 생기는 합계 차이를 보정
- Django Management Command, Bulk Create/Update와 SQL Upsert로 배치 처리

분리한 이유

외부 예측값을 곧바로 구매 데이터에 사용하는 대신 표준화·확인 단계를 두었습니다. 이후 상품 기준의 판매 인벤토리를 생성하여 외부 데이터 수신과 내부 판매 업무를 연결했습니다.

결과

통합·가공한 예측 인벤토리를 판매에 활용하여 기존 판매품 대비 판매 가능한 인벤토리를 20% 이상 확대했습니다. 매출 증가율과는 구분되는 지표입니다.

02 / ADVERTISING PLATFORM

구매·차감과 데이터 정합성

구매 결과와 인벤토리 변경이 함께 반영되도록 트랜잭션으로 처리했습니다.

구매 흐름

01 구매 조건 확인

선택 상품·유닛·기간·금액과 인벤토리 확인



02 업무 데이터 생성

구매유닛 및 하위 광고 데이터 구성



03 인벤토리 차감

상품 특성과 구매 기간·비율에 맞춰 반영



04 결과 반영

구매 흐름을 트랜잭션 단위로 처리

주요 구현 포인트

문제	처리
일부 작업 실패	Django transaction.atomic을 활용해 구매 데이터와 인벤토리 변경의 정합성 유지
분배 합계 불일치	금액·목표량 등의 분배 과정에서 반올림 차이를 보정
상품별 차감 방식 차이	상품 특성에 따라 구매 기간과 인벤토리 기준을 구분
하위 광고 데이터 필요	구매 정보를 바탕으로 운영·송출에 사용할 광고 데이터를 생성

개발 경험

금액과 수량, 기간, 비중이 함께 변하는 업무를 다루며 계산 자체뿐 아니라 저장·실패·복구 시 데이터가 일관되게 유지되는 흐름을 구현했습니다.

02 / ADVERTISING PLATFORM

외부 연동과 운영 기반

업무 기능 구현과 함께 운영에서 필요한 연동·문서·모니터링을 구성했습니다.

외부 시스템 API

연동 대상	개발 내용
광고 메타데이터	외부 협력사 시스템에 전달할 광고 운영 데이터 API 설계·개발
소재 인코딩	인코딩 요청과 상태 동기화, 결과 콜백 처리
외부 예측 인벤토리	수신 데이터를 내부 판매 기준으로 가공하는 처리 흐름

업무 문서 자동화

openpyxl을 활용해 캠페인·상품 데이터 기반의 광고 제안서와 타겟팅 시트를 생성했습니다. 병합 셀, 다중 행, 계산식과 서식을 처리하여 운영 데이터에서 문서로 이어지는 흐름을 구성했습니다.

개발 및 운영 환경

- Docker 기반 개발 환경 구성
- Sentry 도입으로 실시간 에러 추적
- Git 브랜치·PR 규칙과 코드·DB 명명 규칙 수립
- 고객사 피드백을 이슈로 정리하고 기능에 반영

프로젝트에서 보여주는 역할

고객과 업무를 정의하고, DB·서비스·외부 연동을 설계하며, 팀의 개발 일정과 표준을 함께 관리한 상용 시스템 구축 경험입니다.

02 / ADVERTISING PLATFORM

AI 개발 컨텍스트 관리

프로젝트 지식을 공통 관리해 AI 기반 개발의 일관성과 연속성을 높였습니다.

공통 컨텍스트 구성

구성	역할
PROJECT_CONTEXT.md	프로젝트 개요·도메인 구조·핵심 로직·업무 정책·변경 이력과 문서 갱신 규칙을 통합 관리
AGENTS.md / CLAUDE.md	Codex와 Claude Code가 각각의 진입 문서를 통해 동일한 프로젝트 컨텍스트를 참조
Codex / Claude Code Hook	작업 종료 시 실제 코드 변경을 기준으로 공통 컨텍스트를 갱신

AI 컨텍스트 유지 흐름

- 01 프로젝트 지식 통합**
구조·핵심 로직·업무 정책을 공통 문서에 기록
- ↓
- 02 AI 작업에 활용**
Codex와 Claude Code가 동일한 기준을 참조
- ↓
- 03 코드 변경 확인**
작업 종료 Hook에서 실제 변경 내용을 확인
- ↓
- 04 컨텍스트 갱신**
관련 항목과 변경 이력을 다음 작업에 반영

안정성 원칙

실제 코드 변경을 기준으로 관련 항목과 변경 이력만 최소한으로 수정하며, 기존 내용을 임의로 삭제하거나 문서 전체를 다시 작성하지 않도록 구성했습니다.

03 / IMAGE SIMILARITY SEARCH

AI 모델 기반 이미지 유사도 검색 PoC

이미지를 벡터화하고 유사 이미지를 검색하는 백엔드 구조를 검증했습니다.

2026.04 - 2026.05 · 기여도 100%

AI 검색 시스템 설계 / 백엔드 개발

약 75% 절감

벡터 메모리 · float32 대비 int8 적용

100ms 이내

PoC에서 확인한 검색 응답시간

검증 목표

이미지 특징 추출부터 벡터 색인, 유사도 검색 API까지 연결하고 검색 품질·메모리 사용·처리 속도를 함께 고려한 구성을 찾았습니다. 상용 서비스 운영과 구분되는 기술 검증 프로젝트입니다.

직접 맡은 범위

- DINOv2 기반 특징 벡터 추출 파이프라인
- FastAPI와 Elasticsearch kNN 기반 검색 API
- int8 양자화, Redis 캐싱 및 검색 구조 최적화
- Parquet 백업·재색인과 Circuit Breaker 구현

사용 기술

Python · FastAPI · DINOv2 · Elasticsearch kNN / HNSW · Redis · Parquet · Docker Compose

성과 기준

메모리 절감률은 float32 대비 int8 벡터 값의 저장 크기 기준이며, 전체 인덱스 메모리와는 구분됩니다. 검색 응답시간은 PoC 환경에서 확인한 결과입니다.

03 / IMAGE SIMILARITY SEARCH

검색 품질과 자원 효율을 고려한 기술 선택

검색 결과의 정성 평가와 메모리·응답시간을 함께 고려해 구성을 조정했습니다.

실제 이미지로 검색 결과 확인

이미지 특징 추출 모델과 벡터 검색 방식을 조사하고, 실제 이미지에 대한 검색 결과의 시각적 유사성을 정성 평가했습니다. 결과를 직접 확인하면서 검색 품질과 메모리 사용량, 응답시간 사이의 균형을 맞췄습니다.

특징 추출 모델: DINOv2

이미지의 시각적 특징을 벡터로 표현할 모델을 조사하여 DINOv2를 선정했습니다. 추출한 특징 벡터를 검색 파이프라인에 적용하고, 실제 반환된 이미지의 유사성을 확인하며 목적에 맞는 검색 결과를 얻을 수 있는지 검토했습니다.

검색 방식: HNSW 기반 kNN

벡터를 효율적으로 검색할 방법을 조사하는 과정에서 HNSW 기반 근사 최근접 이웃 검색을 검토했습니다. 전체 벡터를 일일이 비교하는 계산 부담을 줄이고 유사한 후보를 빠르게 찾기 위해 Elasticsearch의 kNN 검색으로 구현했습니다.

자원 효율: int8 양자화와 캐싱

int8 양자화로 벡터 메모리 사용을 줄이고, Redis 캐싱으로 반복 검색의 응답시간을 개선했습니다. 양자화에 따른 정밀도 손실 가능성과 메모리 절감 효과를 함께 고려하며 검색 결과를 확인했습니다.

판단 기준

검색 결과의 시각적 유사성은 정성적으로 확인하고, 메모리 사용과 응답시간은 성능 관점에서 살펴보았습니다. 검색 품질과 자원 효율을 함께 고려한 구성을 PoC에 적용했습니다.

03 / IMAGE SIMILARITY SEARCH

색인과 재색인 파이프라인

이미지 추론 결과를 저장하여 재색인 때 같은 계산을 반복하지 않도록 했습니다.

01 이미지 데이터 준비

CSV 검증 · 처리 상태 관리

**02 이미지 다운로드 / 특징 추출**

병렬 다운로드 · 배치 단위 DINOv2 추론

**03 벡터 색인**

384차원 벡터 → Elasticsearch Bulk API

**04 벡터 백업**

특징 추출 결과를 Parquet으로 저장

재색인 경로

Parquet 백업 → 벡터 로드 → Elasticsearch 색인. 이미지 다운로드와 모델 추론을 다시 실행하지 않고 이미 추출한 결과를 활용합니다.

처리 상태와 자원 관리

구성	역할
Redis 상태 관리	대기·처리 중·완료·실패 상태를 구분
병렬 다운로드 / 배치 추론	이미지 I/O와 모델 처리 흐름을 구성
Bulk API	여러 벡터를 묶어 Elasticsearch에 적재
Parquet	고차원 벡터의 백업 및 재색인 데이터로 활용

설계 의도

특징 추출과 검색 인덱스 생성을 분리해 색인을 다시 구성할 때 모델 추론 비용을 반복하지 않도록 했습니다.

03 / IMAGE SIMILARITY SEARCH

검색 API와 장애 대응

검색 속도 최적화와 검색 엔진 장애 시 호출 제어를 함께 구현했습니다.

01 검색 요청

FastAPI에서 이미지 검색 요청 처리

**02 Redis 캐시 확인**

동일 요청의 검색 결과 재사용

**03 벡터 기반 검색**

이미지 특징 벡터로 Elasticsearch kNN 조회

**04 결과 반환**

유사 이미지 목록 제공

Circuit Breaker

상태	동작
CLOSED	정상적으로 검색 엔진 호출
OPEN	검색 엔진 호출 차단, 캐시 또는 빈 결과 반환
HALF_OPEN	시험 요청을 허용하여 정상 응답 여부 확인

검증 결과와 범위

- Redis 캐싱과 검색 구조 최적화로 검색 응답시간 100ms 이내 확인
- Parquet 백업을 통한 재색인 경로 구성
- 검색 엔진 장애 시 반복 호출과 API로의 장애 전파를 제한

핵심 경험

모델 선택, 벡터 색인과 검색 API를 하나의 흐름으로 구현했습니다. 검색 엔진 장애 대응과 재색인 경로까지 포함해 시스템 적용 가능성을 검토했습니다.

03 / IMAGE SIMILARITY SEARCH

확장 가능성: 멀티모달 검색과 RAG

이미지 벡터 검색을 검색 계층으로 활용해 질의응답으로 확장할 수 있습니다.

01 이미지 입력·검색

DINOv2 벡터 추출 → Elasticsearch kNN

**02 연결 정보 조회**

설명·카테고리·업무 문서 메타데이터 결합

**03 LLM 컨텍스트 구성**

검색된 이미지와 근거 정보를 제공

**04 결과 생성**

이미지 기반 질의응답과 참조 근거 제공

추가로 필요한 구성

확장 영역	구성 방향
메타데이터 색인	이미지와 연결된 설명·카테고리·업무 문서를 검색 가능한 형태로 저장
복합 검색	이미지 유사도와 텍스트·메타데이터 조건을 함께 반영
생성·평가	검색 결과를 LLM에 제공하고, 검색 적합성과 답변의 근거 충실도를 분리해 검증

구분

현재 PoC는 이미지 벡터 검색까지 구현한 상태이며, 멀티모달 RAG는 이 검색 구조를 기반으로 한 향후 확장 방향입니다.

04 / AD LOG ANALYTICS

대용량 광고 로그 분석 시스템 신규 구축

광고 이벤트 로그를 운영과 성과 확인에 사용하는 리포트 데이터로 가공했습니다.

2024.03 - 2024.05 · 기여도 100%

데이터 처리 파이프라인 설계 / 백엔드 개발

일 10GB+

S3에 저장된 로그 처리

30~40%

기존 대비 분석 처리속도 개선

문제와 목표

기존 Perl 기반 로그 분석기를 Python/Pandas로 재개발했습니다. 광고 요청·노출·클릭 데이터를 수집하고 가공하여 광고주·매체·인벤토리 등의 기준으로 리포트를 생성했습니다.

직접 맡은 범위

- S3 로그 데이터 수집 및 전처리 파이프라인
- Pandas 기반 집계와 다중 리포트 생성
- 벡터화 연산과 MySQL Bulk Upsert 적용
- 실패 파일의 다음 배치 재처리, 프로세스 락과 Cron 자동화

사용 기술

Python · Pandas · PyMySQL · AWS S3 · MySQL · Docker · Linux Cron

04 / AD LOG ANALYTICS

수집·가공·집계·저장

파일 처리와 데이터 집계, DB 반영을 배치 흐름으로 구성했습니다.

01 로그 수집

AWS S3에 저장된 로그 파일 로드

**02 전처리**

Pandas DataFrame 변환 및 분석 필드 가공

**03 지표 집계**

광고주·매체·인벤토리 등 기준별 리포트 생성

**04 DB 저장**

MySQL Bulk Upsert로 집계 결과 반영

처리속도 개선

적용 방식	목적
Pandas 벡터화 연산	데이터 가공·집계를 배열 연산 중심으로 처리
Bulk Upsert	집계 결과를 묶어 DB에 반영
배치 처리	대량 로그를 파일 단위 처리 흐름으로 관리

결과

기존 분석기 대비 분석 처리속도를 30~40% 개선했습니다. 일 10GB 이상 로그를 처리하는 업무를 자동화하고 광고 성과와 운영 상태 확인에 필요한 리포트를 생성했습니다.

04 / AD LOG ANALYTICS

실패 기록과 배치 재처리

오류가 발생한 파일을 추적하고 다음 실행에서 다시 처리하도록 했습니다.

운영 흐름

01 Cron 실행

정기적으로 분석 배치 시작

**02 프로세스 락 확인**

중복 실행 방지

**03 파일 단위 처리**

로그 가공 · 집계 · 결과 저장

**04 실패 기록 / 재처리**

실패 로그와 에러 내용을 기록하고 다음 배치에서 재처리

구현한 안정성 처리

- 프로세스 락으로 분석 작업의 중복 실행 방지
- 실패 로그와 에러 내용을 남겨 처리 문제 확인
- 실패한 파일을 다음 배치에서 파일 단위로 다시 처리

프로젝트에서의 기여

기존 분석기를 새로운 언어로 옮기는 데서 끝내지 않고, 데이터 가공과 DB 반영 방식을 개선했습니다. 정기 실행과 실패 재처리까지 포함해 운영에서 반복 사용할 수 있는 데이터 처리 흐름을 구축했습니다.

핵심 경험

대량 로그의 수집·가공·집계·저장을 직접 구현하고, 성능 개선과 배치 운영의 안정성을 함께 다뤘습니다.